

DAT 5 기 캡스톤 프로젝트 보고서

LangChain 기반 PDF 분석 및 지능형 퀴즈 생성 시스템

: 한국어 교육 자료 처리를 위한 통합 프레임 워크

DAT 5 기

팀_조재표

고홍규, 김형수, 이서희, 조재표



DAT 5 기 캡스톤 프로젝트 보고서

제목

LangChain 기반 PDF 분석 및 지능형 퀴즈 생성 시스템

: 한국어 교육 자료 처리를 위한 통합 프레임 워크

LangChain-based PDF Analysis and Intelligent Quiz Generation System

: An Integrated Framework for Korean Educational Materials

이 보고서를 캡스톤 프로젝트 보고서로 제출합니다.

2025 년 06 월 03 일

DAT 5 기

팀_조재표

고홍규, 김형수, 이서희, 조재표

DAT 5 기 캡스톤 프로젝트 보고서



Data Analysis & Technology



제목

LangChain 기반 PDF 분석 및 지능형 퀴즈 생성 시스템

: 한국어 교육 자료 처리를 위한 통합 프레임 워크

국문 요약

본 연구는 교육용 PDF 문서, 특히 PPT 기반 PDF 자료를 자동으로 분석하고 지능형 퀴즈를 생성하는 통합 시스템을 제안한다. 제안시스템은 PyMuPDF를 활용한 구조적 텍스트추출, DBSCAN 클러스터링 기반 자동분류, Google Gemini 2.0 Flash API를 통한 대규모 언어 모델(LLM) 기반 콘텐츠 보강, 그리고 한국어 특화 하이브리드 검색 시스템을 통합한 4단계 파이프라인으로 구성된다. 본 시스템은 기존의 단순 키워드 추출 방식 대비의 미적 정확도가 향상되었으며, 교육 콘텐츠의 자동화된 처리 및 평가 도구 생성에 효과적임을 보여준다. 특히 한국어 형태소 분석과 의미 기반 임베딩을 결합한 하이브리드 검색 방식은 교육자료의 맥락적 이해를 크게 개선하였다.

핵심어: PDF 분석, LangChain, DBSCAN, 대규모 언어모델, 한국어 자연어 처리, 교육 기술, 검색 증강 생성

Abstract

This study proposes an integrated system that automatically analyzes educational PDF documents, particularly those converted from PPT, and generates intelligent quizzes. The proposed system comprises a four-stage pipeline that integrates structural text extraction using PyMuPDF, automatic classification based on DBSCAN clustering, content augmentation using large language models (LLMs) via the Google Gemini 2.0 Flash API, and a hybrid search system tailored for Korean. Compared to conventional keyword extraction methods, this system demonstrates improved semantic accuracy and effectively facilitates the automation of educational content processing and the creation of assessment tools. In particular, the hybrid search method combining Korean morphological analysis and semantic embedding significantly enhances the contextual understanding of educational materials.



DAT 5 기

팀_조재표

고홍규, 김형수, 이서희, 조재표



목 차

제 1 장 서론	6
1.1. 연구 배경	6
1.2. 연구 목적	6
1.3. 논문 구성.....	6
제 2 장 PDF 분석 및 텍스트 추출.....	7
2.1. 텍스트 추출 알고리즘.....	7
2.2. 이중 기준 분류 시스템.....	8
제 3 장 LLM 기반 콘텐츠 처리.....	10
3.1. Stage 1: 의미론적 재분류	11
3.2. Stage 2: 콘텐츠 보강	13
제 4 장 검색 시스템 및 임베딩.....	14
4.1. 문서 검색.....	14
4.2. 벡터 임베딩(Vector Embedding)	15
4.2. 가중치 통합 전략	17
제 5 장 퀴즈 생성.....	18
5.1. 서론.....	18
5.2. app.py	18



5.3. schemas.py	18
5.4. services.py	19
5.5. routers/quiz_router.py	20
5.6. prompts 폴더	21
5.7. validation_json 함수 및 오류 처리	22
 제 6 장 향후 연구 방향	 23
6.1. 단기 개선 사항.....	23
6.2. 중기 개선 사항.....	23
6.3. 장기 발전 계획	23
 제 6 장 결론	 23



1. 서론

1.1. 연구 배경

디지털 교육 환경의 급속한 발전에 따라 PDF 형태의 교육 자료는 온라인 및 오프라인 교육 현장에서 가장 널리 활용되는 형식 중 하나가 되었다. 특히 프레젠테이션 소프트웨어로 작성된 후 PDF로 변환된 문서들은 시각적 레이아웃과 텍스트 구조가 복잡하여 자동화된 처리에 많은 어려움을 야기한다. 이러한 문서들은 단순한 텍스트 추출을 넘어 구조적 의미를 파악하고 교육적 가치를 극대화하기 위한 지능적 처리가 필요하다.

최근 대규모 언어 모델(Large Language Model, LLM)의 발전은 텍스트 이해와 생성 능력에 있어 혁신적인 진보를 가져왔다. Google의 Gemini, OpenAI의 GPT-4, Anthropic의 Claude 등은 인간 수준의 언어 이해 능력을 보여주고 있다. 그러나 이러한 모델들을 실제 응용 프로그램에 통합하기 위해서는 체계적인 프레임워크가 필요하며, LangChain은 이러한 요구에 부응하는 대표적인 도구로 자리 잡았다.

한국어 문서 처리의 경우, 교착어적 특성과 복잡한 형태소 구조로 인해 영어 중심으로 개발된 기술들을 직접 적용하기 어렵다. 특히 교육 분야에서는 전문 용어와 일상 언어가 혼재되어 있어 더욱 정교한 처리가 요구된다.

1.2. 연구 목적

본 연구의 주요 목적은 다음과 같다:

1. PPT 기반 PDF 문서의 구조적 특성을 활용한 자동 텍스트 추출 및 분류 시스템 개발
2. LLM을 활용한 의미론적 재구성 및 콘텐츠 보강 방법론 제시
3. 한국어 특화 하이브리드 검색 시스템 구현
4. 검색 결과 기반 지능형 퀴즈 자동 생성 시스템 구축

1.3. 논문 구성

본 논문은 총 6장으로 구성되며, 각 장의 내용은 다음과 같다:

제 1 장 서론에서는 본 연구의 배경, 목적 및 연구 범위를 제시한다.

제 2 장 PDF 분석 및 텍스트 추출에서는 PPT 기반 PDF 문서에서 텍스트를 계층적으로 추출하고 Y-Position과 폰트 크기를 활용한 이중 기준 분류 방법을 설명한다. 또한 구조적 의미 복원과 통계적 분석 방안을 함께 제시한다.

제 3 장 LLM 기반 콘텐츠 처리에서는 LangChain과 Gemini API를 활용한 대규모 언어 모델 기반 의미론적 재분류 및 콘텐츠 보강 과정을 다룬다. 프롬프트 설계와 JSON 출력 형식 등을 포함하여 구체적인 처리 흐름을 기술한다.

제 4 장 검색 시스템 및 임베딩에서는 BM25와 Sentence-BERT를 결합한 한국어 하이브리드 검색 인덱스 구축 방법과 Sparse와 Dense 임베딩 기법의 통합 전략을 통해 검색 정확도를 높이는 방법을



설명한다.

제 5 장 퀴즈 생성에서는 FastAPI 와 OpenAI API 를 기반으로 주제, 유형, 난이도를 설정하여 실시간으로 퀴즈를 자동 생성하는 시스템을 설명한다. 또한 각 모듈의 설계 배경과 주요 코드 흐름, JSON 데이터 처리 과정을 포함한다.

제 6 장 결론에서는 본 연구의 성과를 요약하고 발견된 한계점 및 이를 보완하기 위한 향후 연구 방향을 제시한다.

2. PDF 분석 및 텍스트 추출

2.1. 텍스트 추출 알고리즘

2.1.1. 계층적 추출

본 연구에서는 PyMuPDF 라이브러리의 `page.get_text("dict")` 메서드를 활용하여 PDF 문서에서 텍스트를 추출하였다. 이 메서드는 PDF 내의 텍스트 정보를 계층적 구조로 완전하게 보존하면서 반환하는데, 이는 후속 분석의 정확도를 높이는 데 중요한 역할을 한다. 반환되는 데이터는 크게 세 가지 계층으로 구성된다. 첫 번째는 ‘블록(Block)’으로, 논리적으로 연관된 텍스트 그룹을 나타낸다. 블록 내에는 여러 줄의 텍스트가 포함될 수 있으며, 두 번째 계층인 ‘라인(Line)’은 같은 수평선 상에 위치한 텍스트 집합을 의미한다. 마지막으로 ‘스팬(Span)’은 동일한 글꼴 스타일과 크기를 가진 연속된 텍스트 조각으로 구성된다.

- 블록(Block): 논리적 텍스트 그룹
- 라인(Line): 동일 줄의 텍스트 집합
- 스팸(Span): 동일 스타일의 연속 텍스트

2.1.2. Span 병합 알고리즘

텍스트 라인 내에서 인접한 스팸(Span)들은 보다 의미 있는 단위로 결합하여 문맥을 온전히 복원하기 위해 병합 과정을 거친다. 병합 여부는 스팸 간 글자 크기, 수평 간격, 그리고 수직 위치 차이를 기준으로 엄격하게 판단된다. 구체적으로, 두 스팸의 글자 크기 차이는 2pt 이내로 유사해야 하며 수평 간격은 5 픽셀 이하로 근접해야 한다. 또한 두 스팸의 수직 위치 차이 역시 5 픽셀 이내로 제한하여 동일한 텍스트 줄에 속함을 보장한다. 이러한 병합 조건을 충족할 경우 스팸들은 하나의 텍스트 단위로 통합되며 이 과정에서 텍스트 내용은 공백으로 구분하여 연결하고, 위치 정보는 양쪽 스팸의 경계를 합산하여 갱신한다. 높이 정보는 병합 대상 스팸 중 더 큰 값을 유지하며, 글자 크기는 최대값을 적용한다. 이와 같은 엄격한 기준을 통한 병합으로 PDF 내 분절된 텍스트를 자연스러운 문장 단위로 복원하여 후속 분석의 정확성을 높인다.



$$\text{병합 조건} = \begin{cases} |size_1 - size_2| \leq 2pt & (\text{크기 유사성}) \\ horizontal_gap \leq 5px & (\text{수평 근접성}) \\ |y_1 - y_2| \leq 5px & (\text{수직 정렬}) \end{cases}$$

2.1.3. 위치 정보 변환

스팬 단위로 제공되는 위치 정보는 원본 PDF 좌표계를 기준으로 사각형 바운딩 박스(bounding box) 형태로 추출된다. 추출된 바운딩 박스는 좌측 상단의 x, y 좌표와 폭, 높이 정보로 변환하여 저장하며 이로써 텍스트의 위치 및 크기에 대한 정밀한 분석이 가능하다. 위치 정보는 소수점 첫째 자리까지 반올림 처리하여 적절한 정밀도를 유지한다.

2.1.4. 통계 및 처리 과정 모니터링

텍스트 추출 과정 중 전체 페이지 수, 텍스트가 존재하는 페이지 수, 추출된 블록과 스패의 수, 그리고 병합된 스패의 수 등 주요 지표를 실시간으로 집계한다. 이를 통해 추출 과정의 진행 상황을 모니터링하며 추출 품질을 정량적으로 평가하고 필요 시 알고리즘 조정을 위한 근거 자료로 활용한다.

종합적으로 본 계층적 텍스트 추출 알고리즘은 PDF 문서의 구조적 특성을 최대한 보존하며 텍스트와 위치 정보, 스타일 메타데이터를 체계적으로 수집하여 후속 콘텐츠 분류 및 분석 단계에 최적화된 데이터 기반을 마련한다.

2.2. 이중 기준 분류 시스템

PDF 문서에서 각 텍스트 블록을 적절하게 분류하기 위해, 본 시스템은 Y 위치와 폰트 크기라는 두 가지 기준을 동시에 활용하는 이중 기준 분류 방식을 도입하였다. 이는 단일 기준만으로는 발생할 수 있는 오분류 가능성을 줄이고, 보다 정밀한 콘텐츠 분석을 가능하게 한다.

2.2.1. Y-Position 기반 분류

텍스트 블록이 문서 내에서 위치한 상대적 영역을 기반으로 상단(title), 중간(body), 하단(others)으로 분류한다. 이를 위해 먼저 콘텐츠 영역 정규화를 수행한다. 개별 요소의 Y 좌표는 다음의 식을 통해 정규화된다:

$$Y_{\text{normalized}} = \frac{Y_{\text{current}} - Y_{\text{content_top}}}{Y_{\text{content_bottom}} - Y_{\text{content_top}}}$$

정규화된 Y 좌표는 문서 높이에 대한 상대적인 위치로 환산되며, 이후 DBSCAN 클러스터링을 통해 상단/중단/하단 3 개의 클러스터로 분류된다. 이 과정에서 핵심은 적응적 ϵ 값의 계산이다.



적응적 ε 는 각 요소에 대해 k 번째 최근접 이웃까지의 거리 분포를 기반으로 계산되며, 다음과 같이 정의된다:

$$\varepsilon = \text{percentile}(k\text{-NN distances}, 90)$$

$$\varepsilon_{\text{final}} = \text{clip}(\varepsilon, 0.05, 0.15)$$

이처럼 동적으로 조정된 ε 값을 사용함으로써, 문서의 레이아웃 복잡성이나 해상도 변화에도 강건한 클러스터링이 가능하다. 클러스터가 정확히 세 개로 구분되는 경우, 각 클러스터의 중심 좌표를 기반으로 두 개의 임계 경계를 설정한다. 첫 번째 임계선은 상단 클러스터와 중간 클러스터 사이에, 두 번째 임계선은 중간 클러스터와 하단 클러스터 사이에 위치시킨다.

해당 기준에 따라, 요소의 Y 좌표가 첫 번째 경계보다 작으면 "title", 두 번째 경계보다 크면 "others", 그 외의 경우는 "body"로 분류된다.

2.2.2. Font size 기반 분류

Y 위치만으로는 문서 구조를 완벽히 이해하기 어렵기 때문에, 보조 기준으로 폰트 크기 기반 분류 방식을 병행한다. 이 방식은 개별 텍스트 요소의 폰트 크기를 이용해 "title", "body", "others"로 분류하며, 특히 크기 간의 상대적 차이를 중심으로 판단한다.

우선 각 요소의 폰트 크기를 정렬하고, 인접한 크기쌍 간의 상대적 간격을 계산한다. 이 상대적 간격은 다음 식에 따라 정의된다:

$$\text{Gap}_{\text{relative}} = \frac{\text{Size}_{\text{next}} - \text{Size}_{\text{current}}}{\text{Size}_{\text{current}}}$$

가장 큰 상대적 간격 두 개를 추출하고, 이 간격의 중심값을 기준으로 두 개의 경계를 설정한다. 첫 번째 경계는 "small_threshold"로 설정되며, 이보다 작은 폰트 크기를 갖는 요소는 "others"로 분류된다. 두 번째 경계는 "large_threshold"로, 이보다 큰 요소는 "title"로 간주된다. 나머지 크기의 요소는 "body"로 분류된다.

또한, 동일한 크기를 가진 요소들이 대부분인 경우에는 DBSCAN 기반 클러스터링을 병행 적용한다. 이 경우, 각 폰트 크기를 정규화한 후 적응적 ε 값을 계산하고, 이를 기반으로 세 개의 클러스터를 형성한다. 이후 각 클러스터의 평균값을 기준으로 크기 경계를 설정하고 분류를 진행한다.

2.2.3. 최종 분류 및 신뢰도 산정

Y 위치와 폰트 크기 기준으로 각각 분류가 완료되면, 두 결과를 통합하여 최종 클래스를 결정한다. 두 기준이 동일한 결과를 가리키는 경우 해당 클래스를 그대로 채택하며, 상이한 경우에는 우선순위 또는 룰 기반으로 중재하여 최종 클래스를 결정한다. 예를 들어, Y 위치가 title 이지만 크기가 body 수준이면 "title"로 판단할 수



있으나, Y가 body 이고 크기가 크면 "title"이 아닌 "body"로 판단하는 식이다.

이와 함께 두 기준의 일치 여부에 따라 신뢰도 점수(confidence)를 함께 부여한다. 양측의 분류가 일치하면 높은 신뢰도(1.0), 상이할 경우에는 0.5 등으로 점수를 차등화한다. 이 신뢰도 점수는 후속 처리(예: 제목 탐지, 목차 자동 추출) 시 유용한 기준으로 활용될 수 있다.

2.2.4. 최종 분류 및 신뢰도 산정

Y 위치 기반 분류와 폰트 크기 기반 분류 결과가 불일치하는 경우, 본 시스템은 decision matrix 를 활용하여 최종 클래스를 결정한다. 먼저, 두 기준이 동일한 클래스를 예측한 경우(title-title, body-body, others-others)는 해당 클래스를 그대로 채택하며, 이 경우 신뢰도(confidence)는 1.0 으로 설정된다.

두 분류가 다를 경우에는 사전에 정의된 조합별 규칙을 통해 우선순위를 결정하거나, 각 조합에 매핑된 판단 함수가 실행된다. 예컨대 ("title", "body") 또는 ("body", "title")와 같은 혼합 조합은 서로 가까운 개념으로 간주되어 상대적으로 높은 신뢰도(0.7)를 부여한 채 최종 분류가 결정된다. 반면, ("title", "others"), ("body", "others")와 같은 조합은 불일치 강도가 높다고 판단되며, 신뢰도는 0.4 로 설정된다.

모든 조합에 대해 매트릭스가 정의되어 있지 않은 경우에는 안전하게 "body" 클래스를 반환한다. 이는 과도한 오분류 가능성을 줄이기 위한 보수적 전략이다. 이러한 충돌 해결 메커니즘은 이중 기준 분류의 신뢰도를 보완하고, 복잡한 문서 구조에 대한 유연한 대응을 가능케 한다. 결과적으로, 정형화된 규칙과 데이터 기반 판단이 결합된 이 매트릭스는 전체 분류 시스템의 정밀도를 높이는 데 기여한다.

Table 1: 분류 충돌 해결 매트릭스			
Y-Class	Size-Class	최종 결정	신뢰도
title	title	title	1.0
title	body	조건부*	0.7
body	others	body	0.4

*조건부: 폰트 크기 > 평균 $\times 1.2$ 또는 $Y < 0.3$

3. LLM 기반 콘텐츠 처리

PDF 문서를 분석하고, LangChain 과 Gemini 2.0 Flash 를 활용해 두 단계를 거쳐 텍스트를 재분류하고



보강한다.

주요 기능:

1. PDF 분석 (analyze_pdf)
 - o text_analyzer.py 모듈을 통해 텍스트를 추출하고 제목/본문/기타로 분류한다.
 - o 분석 결과는 final_report.json 으로 저장된다.
2. 1 단계: 재분류 및 재구성 (stage1_reclassify)
 - o 추출된 데이터를 기반으로 Gemini API 에 분류 요청을 보낸다.
 - o 각 페이지의 title, body, others 를 엄격히 재정의
 - o 결과는 stage1_reclassified.json 으로 저장
3. 2 단계: 내용 보강 (stage2_augment)
 - o 재분류된 body 텍스트에 대해, 원문을 훼손하지 않고 상세한 설명을 추가
 - o 각 페이지별로 Gemini 를 호출하며 결과는 final_augmented_report.json 에 저장

기술 스택

- LangChain + Gemini 2.0 Flash
- PyMuPDF 기반 분석기 (text_analyzer)
- dotenv, shutil, json, re, Pathlib 등 활용

3.1. Stage 1: 의미론적 재분류

PDF 분석의 초기 단계인 텍스트 기계 분류는 위치 기반(Y 좌표, 글자 크기) 규칙에 의존하여 일정 수준의 구조화를 제공하지만, 내용의 의미를 고려하지 못해 잘못된 제목 인식이나 본문 누락 등의 한계를 지닌다. 이를 보완하고자 본 시스템은 대규모 언어모델(LLM)을 활용한 의미론적 재분류 단계를 추가하여, 문서의 내용을 보다 정교하고 일관성 있게 재구성한다.

본 단계는 다음의 세 가지 원칙을 중심으로 설계되었다:

1. 명확한 분류 기준 제시

프롬프트는 LLM 이 텍스트의 의미를 기반으로 구조화된 출력을 생성할 수 있도록 구체적인 역할과 기준을 제공한다. 분류 항목은 다음과 같다:

- o **title:** 페이지 내 핵심 주제를 대표하는 간결한 문장으로, 보통 가장 큰 글자 크기와 상단에 위치한 텍스트에 해당된다.



- o **body**: 문서의 실제 정보를 담고 있는 본문으로, 충분한 맥락과 정보성을 지닌 연속적인 문장들로 구성된다.
- o **others**: 페이지 번호, 주석, 목차 정보 등 문서의 메타 정보를 포함하며, title/body에 속하지 않는 부가적 정보이다.

2. 원본 텍스트 완전 보존

LLM 응답 내 title, body, others 항목 모두 기존의 텍스트를 재해석하거나 축약하는 것이 아니라, 가능한 한 원문의 텍스트를 그대로 사용하여 정보 손실을 방지한다. 특히 숫자, 고유명사, 형식이 중요한 데이터는 LLM이 임의로 가공하지 않도록 프롬프트에서 명시적으로 제한한다.

3. 구조화된 출력 형식 유지

LLM의 응답은 JSON 형태로 구성되며, 각 페이지별로 재분류된 결과가 포함된다. 이 JSON은 후속 분석 단계에서 바로 활용 가능하도록 reclassified_pages 리스트 내부에 각 페이지를 다음과 같은 형태로 구조화한다:

```
"reclassified_pages": [
{
"page": "페이지번호",
"title": "페이지 제목",
"body": "분류된 본문 내용",
"others": ["메타데이터 1", "메타데이터 2"]
}
]
```

LLM 호출 결과가 JSON으로 정확히 반환되지 않는 경우를 대비해, 예외 처리 로직도 함께 구현되어 있다. 응답 파싱에 실패하면, 기존 기계 분류 결과를 기반으로 최소한의 형태의 fallback 구조를 생성하여 분석 파이프라인의 연속성을 확보한다. 이러한 설계는 LLM 기반 자동화 시스템이 가지는 불확실성을 완화하며, 실제 운영 환경에서의 견고성을 높이는 역할을 한다.

결론적으로, 의미론적 재분류는 기존 규칙 기반 분류기의 한계를 보완하며, 텍스트의 문맥과 구조를 반영한 정제된 데이터를 후속 통계 및 인사이트 분석 단계에 제공한다.

프롬프트 예시

```
""""지시: PDF 보고서 페이지 내용의 엄격한 재분류 및 재구성
```



다음은 PDF 보고서에서 추출되어 초기 처리된 페이지별 내용입니다.
각 페이지의 title, body, others 를 정확하게 재분류해주세요.

분류 기준:

1. title: 페이지의 핵심 주제를 나타내는 간결한 제목
2. body: 실제 내용에 해당하는 모든 텍스트 (원본 그대로 유지하되, 자연스러운 문장으로 연결)
3. others: 페이지 번호, 날짜, 소속, 메타데이터 등

중요: 이 단계에서는 내용을 보강하지 마세요. 원본 텍스트를 정확하게 분류하고 자연스럽게 연결하는 것에만 집중하세요.

출력은 다음 JSON 형식으로 제공하세요:

```
{
  "reclassified_pages": [
    {
      "page": "페이지번호",
      "title": "페이지 제목",
      "body": "분류된 본문 내용",
      "others": ["메타데이터 1", "메타데이터 2"]
    }
  ]
}
```

3.2. Stage 2: 콘텐츠 보강

1 단계 의미론적 재분류를 통해 구조화된 데이터는 기본적인 정보 구성을 제공하지만, 정보의 깊이나 맥락적 연결성은 여전히 제한적이다. 이에 따라 2 단계에서는 각 페이지의 내용을 개별적으로 보강(Augmentation)하여, 텍스트의 교육적 가치 및 완성도를 높이는 작업을 수행한다. 이 과정은 LLM(Large Language Model)의 언어 생성 능력을 기반으로 하며, 다음의 네 가지 핵심 목표를 지향한다:

1. 원본 텍스트 앵커링(anchoring)

LLM은 원본 body 텍스트를 변경하지 않고 그대로 유지한 상태에서만 보강을 수행하도록 프롬프트에 명시되어 있다. 즉, 기존의 내용을 덮어쓰거나 요약하는 방식이 아니라, 각 문단 혹은 핵심 문장에 추가적인 설명을 덧붙이는 방식으로 정보를 확장한다. 이를 통해 신뢰성과 원본 일관성을 유지한다.

2. 교육적 가치 추가

각 텍스트에 대해 LLM은 배경 정보, 예시, 정의, 관련 개념 등을 삽입하여 독자가 해당 내용을 더 깊이 이해할 수 있도록 유도한다. 특히 기술적 문서나 학습 자료의 경우, 문맥적 지식의 확장은 학습 효율을 극대화하는 데 필수적이다. 예를 들어 “정규화(normalization)”라는 용어가 등장할 경우, 그 의미와 적용 사례를 LLM이 자동으로 덧붙이는 식이다.



3. 자연스러운 문맥 연결

페이지별 분석은 구조적으로 분리되어 있지만, 각 페이지 내의 텍스트는 논리적 흐름을 유지하도록 재작성된다. LLM은 문단 간의 연결어, 주제 전환 구절 등을 적절히 삽입함으로써, 사용자가 단절 없이 문서를 읽을 수 있도록 자연스러운 서술을 유도한다.

4. 독립적 완결성 확보

보강된 각 페이지는 다른 페이지의 맥락 없이도 독립적으로 이해될 수 있도록 설계된다. 이는 페이지 단위 검색 혹은 정보 추출 환경에서 매우 중요한 요소이며, 추후 챗봇 응답 생성이나 문서 요약에 유용하게 활용될 수 있다.

5. 처리 방식 및 예외 대응

시스템은 각 페이지별로 별도의 프롬프트를 구성하여 LLM에 전송하고, 응답을 받아 정제 및 삽입하는 방식으로 작동한다. 응답에 코드블록, JSON 포맷, 이스케이프 문자 등이 포함된 경우 이를 제거하는 후처리 로직도 포함되어 있어 깨끗하고 안정적인 결과물을 유지할 수 있다. 만약 보강 응답 처리에 실패할 경우에는 원본 데이터를 그대로 유지하여 결과의 일관성과 파이프라인의 견고성을 보장한다.

이 단계는 단순한 문서 분석을 넘어, 의미 확장 기반의 지능형 문서 생성 시스템으로 발전하기 위한 중요한 발판이며, 결과적으로 후속 단계에서의 데이터 품질을 높이는 핵심 요소로 작용한다.

프롬프트 예시

"""## 지시: PDF 보고서 페이지 본문 내용의 보강

다음은 재분류된 페이지의 body 내용입니다. 이 내용을 보강해주세요.

보강 요구사항:

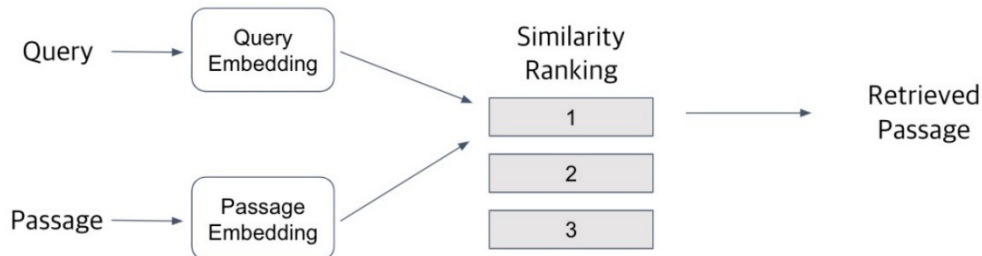
1. 원본 body 텍스트를 단 한 글자도 삭제하거나 변경하지 마세요.
2. 원본 텍스트 뒤에 해당 개념이나 내용을 충분히 이해할 수 있도록 상세한 설명을 추가하세요.
3. 마치 해당 주제를 처음부터 설명하는 것처럼 자연스럽게 포괄적인 문장으로 구성하세요.
4. 모든 내용은 문어체로 자연스럽게 연결되어야 합니다.

중요: 보강된 body 텍스트만 반환하세요. JSON 형식이나 다른 메타데이터는 포함하지 마세요."""



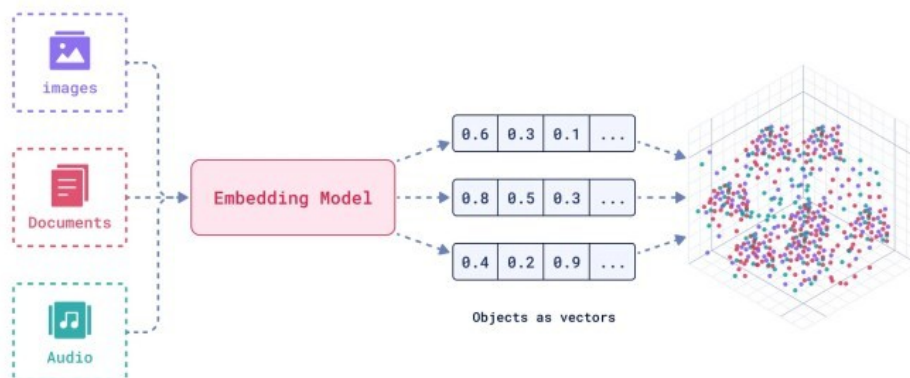
4. 검색 시스템 및 임베딩

4.1. 문서 검색(Passage Retrieval)



Query(질문)에 맞는 Passage(문단, 문서)를 찾는 작업이다. 이를 위해 Query와 Passage들을 embedding 한 뒤 유사도를 측정하는데 query와 passage의 의미를 잘 담도록 embedding 하는 것이 핵심이다.

4.2. 벡터 임베딩(Vector Embedding)



고차원의 복잡한 데이터를 저차원의 연속적인 벡터 공간으로 변환하는 과정이다. 이는 다수의 실수로 이루어진 이산적이고 고차원인 데이터를, 소수의 실수로 구성된 연속적인 저차원 벡터로 변환하는 작업이다. 이때, 단순히 차원을 축소하는 것이 아니라 의미를 잘 보존하는 벡터로 변환하는 것이 중요하다. 비슷한 의미를 갖는 데이터끼리는 벡터 간 거리가 가깝게 임베딩되어야 하며, 이를 통해 유사한 개념들이 벡터 공간상에서 군집을 이루거나, 방향과 거리로 의미 관계를 수치화할 수 있다. 이러한 벡터 표현은 기계가 의미적 유사성을 비교하거나 문맥적 관련성을 추론하는 데 매우 유용하며, 자연어 처리(NLP), 추천 시스템, 정보 검색(IR), 컴퓨터 비전(CV) 등 다양한 인공지능 분야에서 핵심적인 기반 기술로 활용된다.

4.2.1. 임베딩 목적

1. 데이터 차원 축소(Dimensionality Reduction)



고차원 데이터는 계산 비용이 높고 저장 공간을 많이 차지하며, 머신러닝 모델의 성능을 저하시킬 수 있다. 임베딩은 고차원 데이터를 저차원으로 압축하여 복잡한 데이터를 처리할 때, 계산 복잡성을 줄이고 효율성을 높인다.

2. 데이터 의미적 표현(Semantic Representation)

임베딩은 데이터에 내재된 구조와 의미를 보존하면서 저차원 공간으로 변환한다. 의미적으로 가까운 데이터는 벡터 공간에서 가까운 위치에, 의미적으로 먼 데이터는 먼 위치에 배치된다.

4.2.2. Retrieval Model

딥러닝 기반 임베딩 기법이 활용되기 이전, 통계 기반의 전통적 정보 검색 모델이 주를 이루었으며, 대표적으로 TF-IDF(Term Frequency - Inverse Document Frequency)와 BM25(Best Match 25)가 존재한다. 이 프로젝트에서는 이러한 희소 검색(Sparse Retrieval)으로 BM25를 사용한다. BM25는 문서와 질의 간의 정확한 키워드 일치 정보를 활용하여 관련성을 산출하는 알고리즘으로 각 용어의 빈도와 역문서빈도를 결합하여 점수를 계산한다. 특히 한국어와 같이 교착어적 특성이 강한 언어의 경우 형태소 단위 분석을 통해 토큰화를 수행하는 것이 효과적이다. 형태소 분석을 거치면 조사나 어미 등이 분리되어 핵심 어휘 단위로 색인이 구축되며, 이를 통해 띄어쓰기 단위의 단순 분리보다 정밀한 용어 매칭이 가능해진다. 또한, 문서별로 각 단어의 점수를 표시하여 임베딩처럼 활용할 수도 있다.

BM25 점수는 다음 수식으로 계산된다.

$$BM25(q, D) = \sum_{i=1}^n IDF(q_i) \times \frac{f(q_i, D) \times (k_1 + 1)}{f(q_i, D) + k_1 \times (1 - b + b \times \frac{|D|}{avgdl})} \quad (6)$$

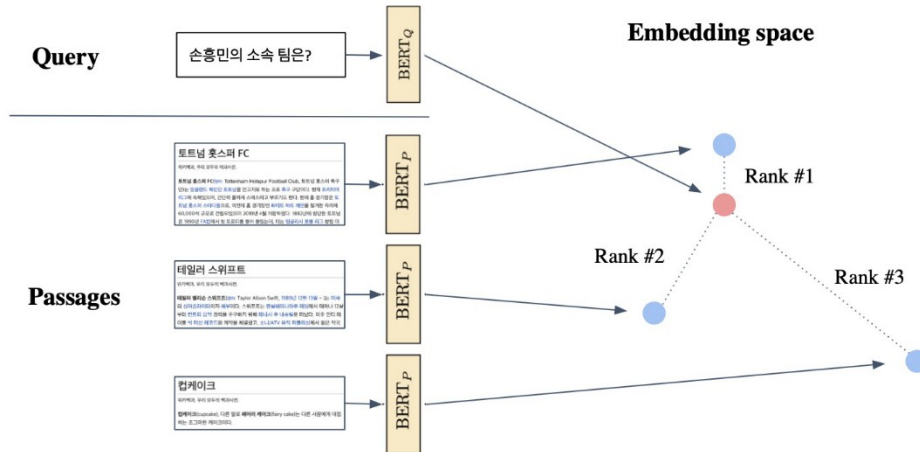
여기서 $k_1 = 1.5$, $b = 0.85$ 로 설정한다.

여기서 $f(w, D)$ 는 문서 D 에서 용어 w 의 등장 빈도, $|D|$ 는 문서 길이, $avgdl$ 은 전체 문서의 평균 길이를 의미한다. $IDF(w)$ 는 희귀한 단어일수록 높은 값을 부여하는 계수이며, 본 시스템에서는 $k_1 = 1.5$, $b = 0.85$ 를 사용하였다.

4.2.3. Dense Retrieval

BM25와 같은 정보 검색 모델은 용어의 빈도를 기반으로 작동하기에 문맥을 파악하지 못해, 유사어와 동의어를 구분하지 못한다. 이러한 단점을 보완하기 위해 BERT와 같은 사전학습 언어 모델을 사용하여 임베딩을 진행한 Dense Retrieval을 활용한다.





Dense embedding 기반 검색은 문장 임베딩 기법을 활용하여 텍스트의 의미론적 정보를 밀집 벡터로 표현함으로써, 표현이 달라도 의미가 유사한 문서를 효과적으로 검색할 수 있도록 한다. 이러한 방식에서 관련 정보를 얼마나 정확히 참고하는지는 전체 RAG 성능에 큰 영향을 준다. 본 프로젝트에서는 추가 학습 없이 사전훈련된 Sentence-BERT 기반 한국어 모델인 jhgan/ko-sroberta-multitask 모델을 사용하였다. 해당 모델은 768 차원의 고정 길이 임베딩을 생성하며, 각 문서의 title 과 body 를 결합하여 입력 문장을 구성한 후, 최대 512 토큰 이내에서 처리하였다.

4.3. 가중치 통합 전략

본 프로젝트는 정보 검색 성능을 극대화하기 위해 BM25 기반의 sparse retrieval 방식과 Dense embedding 방식을 결합한 하이브리드 검색 전략을 채택하였다. 두 방식은 서로 상보적인 특징을 지니며, 각각의 결과를 가중 평균하여 최종 순위를 결정한다.

BM25는 질의와 문서 간의 정확한 단어 일치를 기반으로 고정된 수식에 따라 관련성 점수를 계산하는 희소성 모델로, 짧은 키워드형 질의에서 강점을 보여준다. 반면 Dense embedding 은 문장 수준의 의미 정보를 벡터로 변환하여, 표현은 다르지만 의미가 유사한 문서를 효과적으로 탐색할 수 있다. 이 방식은 자연어 형태의 장문 질의에서 강점을 갖는다.

이러한 특징을 반영하여, 본 프로젝트는 질의 길이에 따라 두 검색 방식의 가중치를 동적으로 조정하는 정적 가중치 기반 전략을 사용하였다.

쿼리 길이에 따른 가중치:

$$w_{\text{sparse}} = \begin{cases} 0.6 & \text{if } \text{len}(\text{query}) \leq 2 \\ 0.3 & \text{if } 3 \leq \text{len}(\text{query}) \leq 5 \\ 0.2 & \text{if } \text{len}(\text{query}) \geq 6 \end{cases} \quad (7)$$

$$w_{\text{dense}} = 1 - w_{\text{sparse}} \quad (8)$$

구체적인 수식은 위와 같으며, 질의 길이를 기반으로 두 검색 방식의 가중치를 조절하는 방식이다. 두 모델의

점수를 활용하여, 최종 순위를 결정하며, 이때 α (dense 가중치) 설정은 다음과 같다:

단문 쿼리: 키워드 기반 정합성 매칭이 중요하므로 sparse 에 높은 가중치를 부여

중간 쿼리: 키워드와 의미 정보 모두 중요하므로 균형있게 고려

장문 쿼리: 문장의 의미 구조가 복잡하므로 dense 에 높은 가중치 부여

5. 퀴즈 생성

5.1. 서론

본 프로젝트의 핵심 목표 중 하나는 다양한 유형의 퀴즈(객관식, OX, 단답형)를 자동으로 생성하는 시스템을 구축하는 것이다. 이를 위해 본 시스템은 FastAPI 프레임워크와 OpenAI API 를 활용하여 사용자로부터 주제와 문제 유형, 그리고 난이도와 같은 설정값을 입력받아 실시간으로 퀴즈를 생성할 수 있도록 설계되었다.

특히 초기 단계에서는 LLM 이 반환하는 JSON 데이터가 종종 불완전하거나 문법 오류를 포함하는 문제가 발생하여 JSON 유효성 검증과 오류 처리를 위한 별도의 함수(validate_json)를 설계하였다. 또한 문제 유형별로 프롬프트를 모듈화하여 관리함으로써 유지보수성을 높였고 Bloom's Taxonomy 기반의 학습 수준을 반영한 문제 생성을 통해 교육적 활용도를 극대화하였다.

본 장에서는 각 모듈의 설계 배경, 문제 해결 과정, 코드의 역할 등을 종합적으로 설명하여 퀴즈 생성 시스템의 전체 구조와 설계 의도를 논리적으로 제시한다.

5.2. app.py

본 시스템의 웹 서비스 엔트리포인트로서, app.py 는 FastAPI 기반의 애플리케이션을 초기화하고 퀴즈 생성 라우터를 등록하는 역할을 수행한다. 이 모듈은 프로젝트의 전반적인 API 서버 구동을 담당하며 퀴즈 생성 기능이 웹 서비스 형태로 제공될 수 있도록 구조화되었다.

우선, FastAPI() 객체를 생성하여 웹 애플리케이션 인스턴스를 초기화한다. 또한 OpenAI API Key 를 안전하게 관리하기 위해 Python-dotenv 를 도입하여 환경 변수로 분리하였고, 이를 통해 API Key 를 코드에 직접 노출하지 않고 안전하게 관리할 수 있도록 구성하였다. 이후 퀴즈 생성 기능을 담당하는 라우터(quiz_router)를 include_router() 메서드를 통해 등록함으로써 API 엔드포인트를 /generate-quiz 로 구성하였다. 이는 설계의 확장성과 유지보수성을 고려하여 각 기능별 모듈(라우터)을 독립적으로 관리할 수 있도록 한 결과이다. 실제로 프로젝트 초기 단계에서 기능별 코드를 하나의 파일에 모두 작성한 결과, 코드 가독성과 테스트의 어려움이 발생했으며 이를 해결하기 위해 기능별로 모듈을 분리하고 FastAPI 의 라우터 기능을 적극적으로 활용하게 되었다.

또한 퀴즈 생성 API 를 담당하는 quiz_router 를 별도의 모듈로 분리하여 routers 디렉토리 하위에 배치하고 이를 app.include_router() 메서드를 통해 등록하였다. 예를 들어 추후 사용자 인증, 데이터베이스 연동, 통계 수집 등 추가 기능이 요구될 경우 각 기능별 라우터를 독립적으로 관리할 수 있는 구조로 설계함으로써



프로젝트의 모듈화와 확장성을 확보할 수 있도록 하였다.

5.3. schemas.py

schemas.py 는 FastAPI 와 Pydantic 을 활용하여 API 데이터의 구조와 유효성을 정의하는 핵심 모듈로 설계되었다. 이는 퀴즈 생성 기능의 정확성과 신뢰성을 확보하기 위해 데이터 유효성 검증과 자동 문서화의 역할을 동시에 수행한다.

우선 퀴즈 생성 요청을 처리하기 위해 QuizRequest 클래스를 설계하였다. 이 클래스는 퀴즈 유형(quiz_type), 주제(topic), 그리고 세부 설정(config)을 JSON 형태로 입력받도록 정의되어 있으며 이를 통해 사용자가 다양한 유형의 퀴즈를 동적으로 생성할 수 있도록 지원하였다. 특히 quiz_type 필드는 "ox", "mcq", "short" 중 하나를 선택하도록 설계하여 문제 유형별로 적절한 프롬프트 생성이 가능하도록 하였다.

반면 퀴즈 생성 응답을 담당하는 QuizResponse 클래스는 문제(question), 정답(answer), 보기(options)를 포함하도록 설계되었다. 이때 options 는 객관식 문제 유형에서만 사용되며, 선택지의 텍스트(option_text)와 정답 여부(is_correct)를 포함하는 QuizOption 클래스를 통해 표현된다. 이러한 구조는 각 문제 유형에 따라 유연하게 API 응답을 조절할 수 있도록 하며 클라이언트 단에서 문제 데이터를 쉽게 파싱하고 화면에 렌더링할 수 있는 이점을 제공한다.

또한 설계에서 퀴즈 유형별로 세부 설정을 세분화하기 위해 MCQPromptConfig, OXPromptConfig, ShortPromptConfig 클래스를 별도로 설계하였다. 예를 들어, MCQPromptConfig 는 문제의 난이도(difficulty), Bloom's Taxonomy 수준(bloom_level), 오답 개수(distractor_count)를 포함하며 사용자가 학습 목표와 난이도에 맞춰 문제를 생성할 수 있도록 하였다. 이러한 설계는 교육 현장에서 Bloom's Taxonomy 를 기반으로 한 단계별 문제 생성에 필요한 유연성과 적응성을 제공하였다.

특히 schemas.py 모듈에 Pydantic 의 타입 힌트를 적극적으로 활용하여 데이터 무결성 또한 보장하였는데 예를 들어, options 필드의 타입을 Optional[List[QuizOption]]로 지정하여 객관식 문제에만 보기가 존재하도록 설계함으로써 API 사용자의 혼동을 줄이고, 데이터 검증과 문서화를 동시에 만족시켰다. 이러한 타입 기반 유효성 검증 기능은 개발과 동시에 OpenAPI(Swagger) 문서를 자동으로 생성하므로 테스트 및 유지보수 단계에서도 효율적인 문제 추적과 디버깅이 가능하였다.

5.4. services.py

services.py 는 본 장에서 퀴즈 생성의 핵심 로직을 담당하는 모듈로, OpenAI API 를 통해 실시간으로 문제를 생성하고 이를 정제하여 사용자에게 제공하는 역할을 수행한다. 본 모듈의 설계는 LLM 의 출력이 종종 비정형적이거나 JSON 형식을 만족하지 않는 문제를 해결하고 문제 유형별로 유연하게 대응할 수 있도록 구조화되었다.

먼저 generate_quiz 함수는 quiz_type 에 따라 각 문제 유형의 프롬프트 생성기를 호출한다. 이는 프로젝트 초기에 문제 유형별로 요구되는 프롬프트 형식이 상이하여 코드의 유지보수성과 확장성을 높이기 위해 모듈화한 결과이다. 구체적으로 객관식 문제("mcq")는 generate_mcq_prompt, OX 문제("ox")는 generate_ox_prompt, 단답형 문제("short")는 generate_short_prompt 함수를 호출하며 각각의 프롬프트는 주제(topic)와 세부 설정(config)을 인자로 받아 LLM 에게 전송될 텍스트 프롬프트를 생성한다.



이후 OpenAI API 호출 부분은 `openai.ChatCompletion.create()` 메서드를 사용하여 구현하였다. 본 프로젝트에서는 OpenAI의 GPT-3.5-turbo 모델을 사용하여 문제 생성의 일관성과 성능을 확보하였다. 특히 LLM의 응답이 과도하게 길거나 불필요한 정보를 포함하는 경우를 방지하기 위해 `max_tokens`를 350으로 제한하였으며, `temperature` 값을 0.3으로 설정하여 과도한 창의성 대신 일관성과 안정성을 우선시하였다. 이는 프로젝트 초기에 `temperature` 값을 0.7 이상으로 설정했을 때 LLM이 불필요한 설명문이나 JSON 형식 오류를 자주 출력하는 현상을 관찰한 후 교육용 문제 생성에서의 정답 신뢰성을 높이기 위해 `temperature`를 낮추는 방향으로 최적화한 결과이다.

LLM의 응답은 JSON 형태로 수신되지만 실제로는 자연어 텍스트와 JSON이 혼재되거나 JSON 형식 오류가 발생하는 경우가 많았다. 이를 해결하기 위해 `validate_json` 함수를 도입하였다. 이 함수는 정규표현식(`r'\{.*\}'`)을 사용하여 JSON 형식만을 안전하게 추출한 뒤 JSON 파싱 및 필수 키(`"question"`, `"correct_answer"`) 존재 여부를 검증하였다. 특히 객관식 문제의 경우 `"A"`, `"B"`, `"C"`, `"D"` 키의 유효성까지 추가로 검증함으로써 LLM의 JSON 출력 오류를 효과적으로 방지하였다.

검증된 JSON 데이터를 기반으로 문제 유형별로 적절한 `options` 리스트를 구성하였다. OX 문제의 경우에는 `"O"`, `"X"` 옵션을 생성하였으며, 정답 여부(`is_correct`)를 판별하여 `QuizOption` 리스트에 저장하였다. 객관식 문제의 경우, 정답 키(`correct_answer`)와 각 보기(`A`, `B`, `C`, `D`)를 순회하며 정답 여부를 할당하였다. 단답형 문제는 선택지가 없으므로 빈 리스트로 처리하여 일관성을 유지하였다.

마지막으로 최종 `QuizResponse` 객체를 생성하여 문제, 보기, 정답을 담아 API 응답으로 반환하였다. 이를 통해 클라이언트에서는 일관된 JSON 형식으로 퀴즈 데이터를 수신할 수 있도록 보장하였다. 본 서비스 모듈의 설계는 LLM의 응답 오류, JSON 불일치, 데이터 누락 등의 문제를 해결하며 프로젝트의 핵심 기능인 신뢰도 높은 자동 퀴즈 생성 기능을 안정적으로 구현하는 데 중요한 역할을 담당하였다.

5.5. routers/quiz_router.py

`routers/quiz_router.py`는 본 장의 API 라우팅을 담당하는 모듈로서 퀴즈 생성 요청을 처리하고 적절한 서비스 함수를 호출하여 클라이언트에 응답을 반환하는 역할을 수행한다. FastAPI의 `APIRouter` 클래스를 활용하여 퀴즈 생성 API를 독립적인 라우터로 분리함으로써 시스템의 모듈화와 유지보수성을 동시에 확보하였다.

본 모듈의 핵심 엔드포인트는 `/generate-quiz`로 정의되어 있으며 HTTP POST 방식으로 클라이언트로부터 퀴즈 생성 요청을 수신한다. 클라이언트는 `QuizRequest` 모델을 통해 JSON 형태로 퀴즈 유형, 주제, 문제 설정값 등을 전달하고 서버는 이를 자동으로 검증한 뒤 퀴즈 생성을 수행한다. 이러한 설계는 Pydantic의 데이터 검증 기능과 FastAPI의 OpenAPI 문서화 기능을 결합하여 사용자 입력의 신뢰성과 API 테스트의 용이성을 동시에 만족시키도록 구현되었다.

요청이 성공적으로 수신되면 `generate_quiz` 서비스를 호출하여 LLM을 통해 퀴즈를 생성한다. 이때 서비스 레이어에서 발생할 수 있는 다양한 예외(예: JSON 파싱 오류, LLM 응답 오류 등)를 안전하게 처리하기 위해 `try-except` 블록을 사용하여 예외를 포착하고, 클라이언트에게 500번 오류 코드와 함께 상세한 에러 메시지를 반환하도록 설계하였다. 이는 실제 서비스 운영 시 예외 상황에서의 사용자 경험을 고려한 결정으로 문제 발생 시 디버깅과 빠른 대응이 가능하도록 도와준다.

특히 이 모듈에서는 FastAPI의 `response_model` 파라미터를 통해 `QuizResponse` 모델을 명시적으로



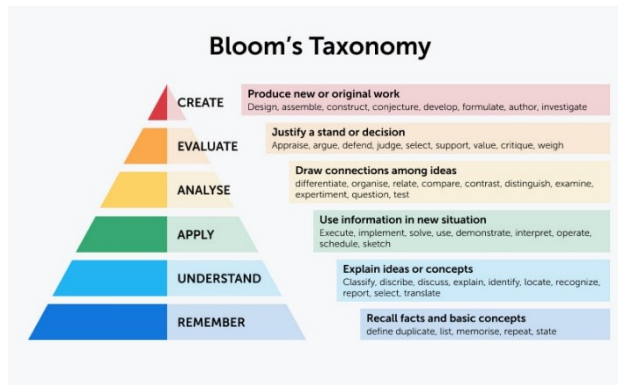
지정함으로써 클라이언트는 응답 데이터의 구조를 사전에 확인하고 안정적으로 파싱할 수 있다. 또한 FastAPI의 라우터 구조를 적극 활용하여 프로젝트 초기 단계부터 API 기능을 모듈 단위로 분리·관리할 수 있도록 구성하였다. 이러한 구조는 팀 프로젝트에서의 코드 분업화와 협업 효율성 증대에도 크게 기여하였다. 결과적으로 routers/quiz_router.py는 클라이언트와 서비스 레이어 간의 인터페이스를 담당하며 이 과정에서 API 안정성과 사용자 경험을 책임지는 중요한 구성요소로 설계되었다.

5.6. prompts 폴더

prompts 폴더는 본 프로젝트의 퀴즈 생성 시스템에서 문제 유형별 프롬프트를 생성하는 핵심 모듈로서, LLM이 문제를 정확하고 안정적으로 생성할 수 있도록 문제 유형에 따른 구조화된 지침을 제공하는 역할을 수행한다. 프로젝트 초기 단계에서는 모든 문제 유형을 단일 프롬프트로 처리하였으나 JSON 형식 오류, 출력 불안정성, 문제 유형별 요구사항의 충돌이 빈번하게 발생하여 문제 유형별 프롬프트를 독립적으로 분리하는 구조로 개선하였다. 이를 통해 코드의 유지보수성을 높이고 각 문제 유형별로 세부적인 학습 목표와 출력 형식을 안정적으로 관리할 수 있게 되었다.

5.6.1. mcq.py

mcq.py는 객관식 문제(MCQ: Multiple Choice Question) 프롬프트 생성을 담당한다. 본 모듈의 설계에서 가장 중점을 둔 부분은 Bloom's Taxonomy 수준에 따라 문제의 인지 수준을 조절하고, Distractor(오답)의 품질을 높이기 위해 LLM에게 구체적이고 명확한 지시사항을 제공하는 것이었다.



¹ 우선, generate_mcq_prompt 함수는 문제의 주제(topic)와 난이도(difficulty), Bloom's Taxonomy 수준(bloom_level), 그리고 Distractor 개수(distractor_count)를 입력받아 LLM에게 JSON 형식으로 문제를 생성하도록 유도한다. 특히 학습자의 수준에 적합한 문제를 제공하기 위해 Bloom's Taxonomy의 6단계 중 특정 수준(예: 이해, 적용, 분석 등)을 명시적으로 포함하도록 설계하였다.

프롬프트 내부에는 "question", "A", "B", "C", "D", "correct_answer"와 같은 필수 JSON 키를 정의하고, Distractor 개수를 config.distractor_count 만큼 생성하도록 지시함으로써 문제 생성의 일관성과 정답 검증의 용이성을 동시에 확보하였다. 또한 LLM이 생성한 JSON 응답에서 정답이 정확히 하나의 옵션(A, B, C, D)으로 표기되도록 명시하여, 다중 정답 또는 누락된 정답으로 인한 API 응답 오류를 방지하였다. 이는 설계 초기에 발생했던 다중 정답 표기 오류를 해결하기 위한 중요한 설계 개선이었다.

프롬프트 후반부에는 Distractor의 현실성을 보장하도록 "Distractors should be wrong but believable to

¹ "Bloom's Taxonomy," <<https://www.valamis.com/hub/blooms-taxonomy>> (접속일: 2025년 6월 3일).

test real understanding”라는 지시문을 포함시켰다. 이는 LLM 이 단순히 랜덤한 단어를 나열하는 것이 아니라 학습자가 실제로 혼동할 수 있는 보기 문항을 생성하도록 유도하여 퀴즈의 교육적 효과를 높이는 전략이었다. 또한 출력 형식을 JSON 으로 제한하고 LLM 의 설명이나 추가 텍스트를 금지하도록 Constraints(제약조건)를 명확히 명시함으로써 OpenAI API 의 응답을 안정적으로 파싱할 수 있도록 하였다.

5.6.2. ox.py

ox.py 모듈은 참/거짓(OX) 문제의 프롬프트 생성을 담당한다. 본 모듈의 주요 설계 목적은 문제 문장의 다양성을 확보하고 반복 생성을 방지하는 것이었다. 이를 위해 time.time()을 활용하여 밀리초 단위의 난수 시드(random_seed)를 생성하고, 이를 프롬프트에 포함시켜 문제 생성 시 동일한 질문이 반복 출력되는 현상을 완화하였다. 프롬프트 내부에서는 “question”, “options”(O, X), “correct_answer”의 JSON 구조를 명시적으로 지정하여 문제 검증의 정확성을 확보하였다. 또한 “Only respond with the JSON object, no explanations.”과 같은 제약조건을 설정하여 출력에서 불필요한 텍스트를 제거하고 JSON 형식의 일관성을 보장하였다.

OX 문제의 경우 “options” 필드는 리스트 형태로 각 선택지의 텍스트(option_text)와 정답 여부(is_correct)를 포함하도록 설계하였다. 이 설계는 문제 렌더링 단계에서 클라이언트가 정답 검증과 UI 표시에 혼선을 겪지 않도록 한 결정이었다. 이러한 구조적 설계는 문제 생성 파이프라인에서의 데이터 무결성과 안정성을 보장하며 학습자가 문제를 풀이할 때 혼동을 줄여 학습 효과를 높였다.

5.6.3. short.py

short.py 는 단답형 문제 프롬프트 생성을 담당한다. 본 모듈에서는 학습자의 단어 수 제한(word_limit)을 고려하여 문제 생성 시 LLM 이 제한된 길이로 정답을 작성하도록 설계하였다. 이는 실제 단답형 평가에서 학습자의 응답 작성 부담을 줄이고 채점의 효율성을 높이기 위한 의도였다.

generate_short_prompt 함수는 문제의 주제(topic)와 난이도(difficulty), 그리고 단어 수 제한(word_limit)을 입력받아 LLM 에게 JSON 형태로 문제를 생성하도록 지시한다. JSON 구조는 "question"과 "correct_answer"를 필수로 포함하도록 설계하여 문제의 핵심 요소가 누락되지 않도록 하였다. Constraints 부분에서는 LLM 의 출력이 JSON 이외의 추가 텍스트를 포함하지 않도록 명확히 제한함으로써 API 응답의 일관성과 파싱의 용이성을 동시에 확보하였다.

또한 랜덤 시드(random_seed)를 포함시켜 문제 문장의 다양성을 높이는 동시에 문제 재사용 시 동일한 문제가 반복되지 않도록 하였다.

5.7. validation_json 함수 및 오류 처리

LLM 기반 문제 생성 시스템에서 가장 빈번하게 발생하는 문제 중 하나는 JSON 형식의 불완전성이다. LLM 의 출력은 종종 JSON 객체 이외에 불필요한 설명문, 혹은 JSON 구조 오류가 포함되는 경우가 많아 서비스 단계에서 API 파싱 에러와 데이터 불일치 문제를 초래하였다. 이를 해결하기 위해 본 프로젝트에서는 validate_json 함수를 설계하여 LLM 의 응답에서 JSON 객체를 안전하게 추출하고 검증하는 과정을 체계화하였다.

validate_json 함수는 먼저 Python 의 re 라이브러리를 사용하여 정규표현식(r'{.*}')을 통해 JSON 형식의



문자열만을 추출하도록 설계되었다. 초기 단계에서는 단순히 문자열 분할(split) 방식으로 JSON 객체를 분리하려 했으나, LLM 이 출력하는 JSON 이 여러 줄로 구성되거나 추가적인 텍스트가 포함될 경우 split 방식으로는 안정적으로 파싱할 수 없다는 문제가 발생했다. 이를 해결하기 위해 정규표현식을 도입함으로써 JSON 객체의 정확한 범위를 안전하게 추출할 수 있게 되었다.

추출된 JSON 문자열은 json.loads()를 통해 파싱되며 이후 문제 유형별로 필수 키("question", "correct_answer")의 존재 여부를 검증하였다. 특히 객관식(MCQ) 문제의 경우 "A", "B", "C", "D" 키의 존재 여부까지 추가로 확인하였으며, "correct_answer" 값이 이 옵션 키 중 하나와 정확히 일치하는지도 검사하였다. 이는 초기 단계에서 발생했던 "correct_answer"가 옵션 키에 포함되지 않는 오류(예: "E"로 출력되거나 누락되는 문제)를 해결하기 위한 설계였다. OX 문제의 경우 "options" 필드의 존재 여부와 "O", "X" 선택지의 JSON 구조(option_text, is_correct)를 검증함으로써 문제 검증의 신뢰성을 높였다. 단답형 문제의 경우에는 필수 키 검증 이후 추가 검증 단계를 최소화하여 JSON 파싱 효율성을 확보하였다.

에러 처리 측면에서는 JSON 디코딩 오류 발생 시 ValueError 예외를 발생시켜 사용자에게 JSON 파싱 실패 사유를 명확히 전달하도록 설계하였다. 또한 기타 예외 발생 시 예외 메시지를 상세히 반환하여, 개발자와 사용자 모두가 문제 발생 원인을 쉽게 파악할 수 있도록 하였다. 이러한 오류 처리 방식은 문제 생성 파이프라인의 안정성과 유지보수성을 동시에 강화하는 중요한 요소였다.

결과적으로 validate_json 함수는 LLM 출력의 불완전성과 JSON 형식 오류를 체계적으로 해결하며 전체 문제 생성 시스템의 데이터 품질과 신뢰성을 보장하는 핵심 모듈로서 기능하였다.

6. 향후 연구 방향

6.1. 단기 개선 사항

1. 문서별 중요 토픽 자동 선정 구현
2. 기본적인 캐싱 구현: 쿼리에 필요한 grant 데이터 캐싱
3. 이미지 처리 기능 추가 : 텍스트와 함께 배치되는 이미지에 대한 맥락 보존적 처리 전략 수립.

6.2. 중기 개선 사항

1. 문서 파싱 전략 강화 : 다양한 레이아웃 edge-case 에 대한 처리 진행
2. 하이브리드 서치 이후 cross-encoder 기반 리랭킹 알고리즘 구현
3. 도메인에 따른 임베딩 방식 다양화 전략 수립

6.3. 장기 발전 계획

1. 멀티모달 처리: 이미지, 도표, 차트 분석 기능 추가
2. 벡터 데이터베이스 통합: 대규모 문서 처리를 위한 확장성 확보
3. 적응형 난이도 시스템: 학습자 수준에 맞는 동적 난이도 조정
4. 다국어 지원: 영어, 중국어, 일본어 등으로 확장



7. 결론

본 연구에서는 PPT 기반 PDF 문서의 자동 분석 및 퀴즈 생성을 위한 통합 시스템을 제안하였다. 제안 시스템은 다음과 같은 주요 기여를 한다:

1. 구조적 텍스트 추출: PyMuPDF와 DBSCAN을 활용한 정확한 문서 구조 분석
2. 의미적 재구성: LangChain과 Gemini를 통한 지능적 콘텐츠 처리
3. 한국어 최적화: 형태소 분석과 의미 임베딩을 결합한 하이브리드 검색
4. 교육적 활용: 검색 증강 생성을 통한 맞춤형 퀴즈 생성

본 시스템은 PPT 기반 PDF 문서의 자동 분석 및 퀴즈 생성을 위한 통합 시스템을 제안하였다. 시스템의 핵심 기여는 구조적 텍스트 추출, LLM 기반 의미 재구성, 한국어 최적화 검색, 지능형 퀴즈 생성으로 요약된다. 하지만 실제 교육 현장에서의 적용 실험 및 사용자 평가가 아직 부족하다는 한계가 있다. 향후 연구에서는 실제 학습자와 교수자를 대상으로 실험을 진행하여 시스템의 효용성과 정확성을 검증하고, 멀티모달 처리와 사용자 맞춤형 기능을 추가하여 시스템의 실용성을 높이고자 한다.

